Introduction to the Theory of Computation

Lecture 20. NP-Complete Problems

Neo (Yuan) Li

Nov 2026

Fudan University

yuan_li@fudan.edu.cn

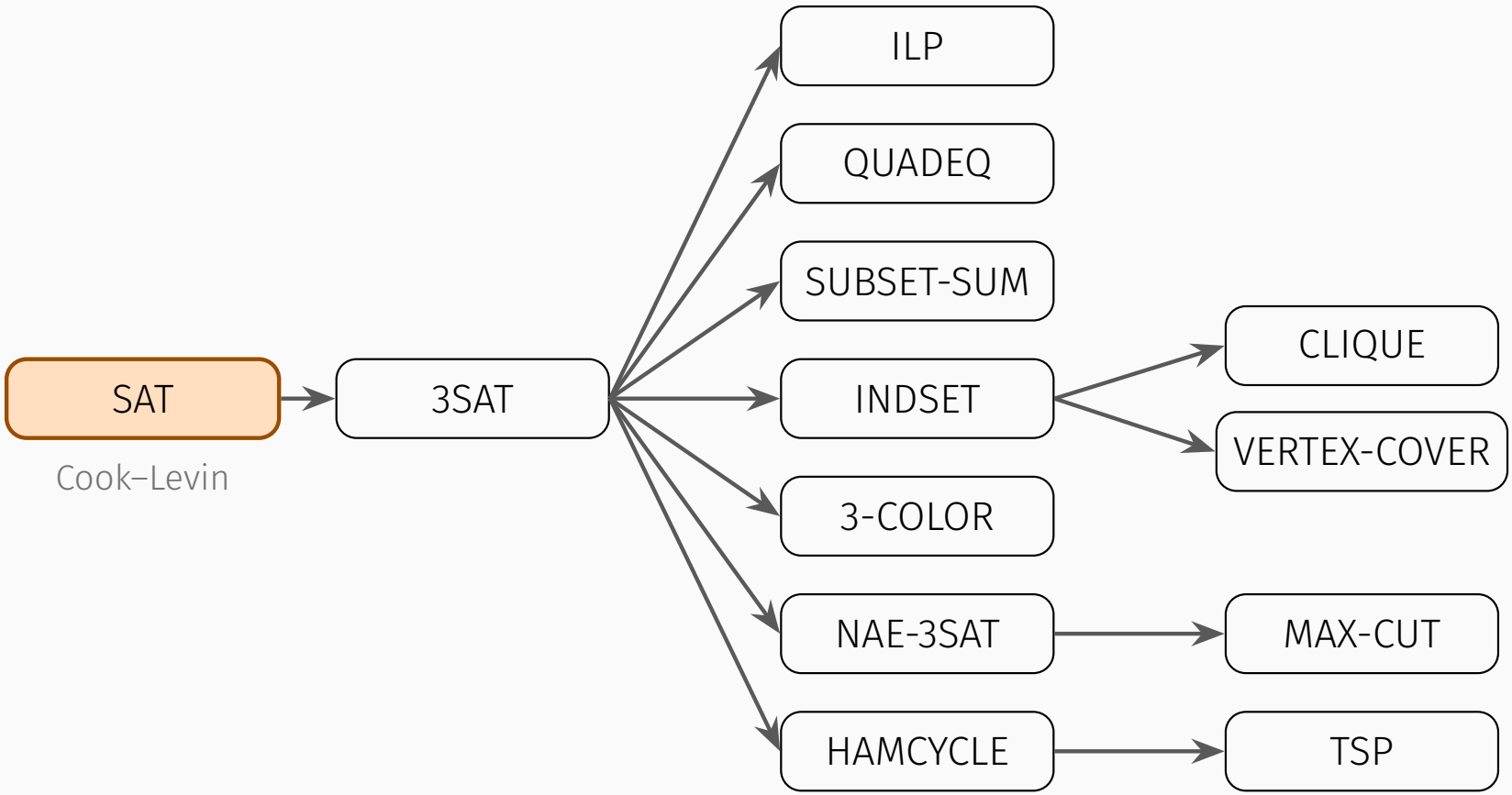
A Wonder of Complexity Theory: Complete Problems

- Thousands of natural problems are **NP-complete**.
- All **poly-time equivalent** under Karp reductions.
- Solve one efficiently $\implies$ solve all of NP.
- **Takeaway:** seek **approximation**, not exact optimum.
- NP-complete $\neq$ hopeless in practice.

A Brief History of NP-Completeness

- **Cook (1971)**: SAT is NP-complete.
- **Levin (1973)**: same result, independently.
- **Karp (1972)**: 21 problems NP-complete.
- **Garey–Johnson (1979)**: *Computers and Intractability* — hundreds more cataloged.
- **Today**: many thousands.

Reduction Web



3SAT

A **3CNF** formula is a CNF formula in which every clause has exactly 3 literals, e.g.

$$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_4).$$

Definition 1
$3\mathrm{SAT} = \{\varphi : \varphi \text{ is a satisfiable 3CNF}\}$.

Theorem 2
$3\mathrm{SAT}$ *is NP-complete.*

Proof Strategy

- $\mathrm{3SAT} \in \mathrm{NP}$: special case of SAT.
- NP-hard: show $\mathrm{SAT} \leq_p \mathrm{3SAT}$.

The Splitting Gadget

For $C = \ell_1 \vee \cdots \vee \ell_k$ ($k \geq 4$), with new variables $z_1, \ldots, z_{k-3}$:

$$C' = (\ell_1 \vee \ell_2 \vee z_1) \wedge (\neg z_1 \vee \ell_3 \vee z_2) \wedge \cdots \wedge (\neg z_{k-3} \vee \ell_{k-1} \vee \ell_k).$$

Claim: σ satisfies $C \iff \sigma$ extends to the z's satisfying C'.

Example of the Gadget

$$x_1 \vee x_2 \vee x_3 \vee x_5 \quad \rightsquigarrow \quad (x_1 \vee x_2 \vee z) \wedge (\neg z \vee x_3 \vee x_5)$$

- $x_1 \vee x_2$ true: set $z = 0$.
- $x_3 \vee x_5$ true: set $z = 1$.
- All four false: left with $(z) \wedge (\neg z)$.

Integer Linear Programming

Definition 3

$$\mathrm{ILP} = \{\, \langle A, \mathbf{b} \rangle : A \in \mathbb{Z}^{m \times n},\ \mathbf{b} \in \mathbb{Z}^m,\ \exists\, \mathbf{x} \in \mathbb{Z}^n \text{ with } A\mathbf{x} \leq \mathbf{b} \,\}.$$

($\geq$ and $=$ constraints are allowed: multiply by -1, or use two inequalities.)

Yes instance

$$\begin{cases} x_1 + x_2 \leq 3 \\ x_1 \geq 1 \\ x_2 \geq 2 \end{cases}$$

$\mathbf{x} = (1, 2)$. ✓

No instance

$$\begin{cases} 2x_1 - 2x_2 \leq 1 \\ 2x_1 - 2x_2 \geq 1 \end{cases}$$

Forces $x_1 - x_2 = \frac{1}{2}$. ×

ILP Is NP-Complete

Theorem 4
ILP *is NP-complete.*

- $\mathrm{ILP} \in \mathrm{NP}$: certificate $\mathbf{x}$. (If feasible, some solution has poly-size entries; Papadimitriou, 1981.)
- NP-hard: show $3\mathrm{SAT} \leq_p \mathrm{ILP}$.

$3\mathrm{SAT} \leq_p \mathrm{ILP}$

Given a 3CNF φ over $x_1, \ldots, x_n$:

- **Variables:** $0 \leq x_i \leq 1$, so $x_i \in \{0, 1\}$.
- **Clauses:** literal $x_i \mapsto x_i$, $\neg x_i \mapsto 1 - x_i$; require sum ≥ 1.

$$x_1 \vee \neg x_2 \vee \neg x_3 \;\rightsquigarrow\; x_1 + (1 - x_2) + (1 - x_3) \geq 1.$$

Claim: φ is satisfiable $\iff$ the ILP is feasible.

Quadratic Equations over $\mathrm{GF}(2)$

Variables $x_1, \ldots, x_n \in \mathrm{GF}(2)$

Definition 5
$\mathrm{QUADEQ} = \{\langle E \rangle : E$ is a system of equations of degree ≤ 2 over $\mathrm{GF}(2)$ with a solution$\}$.

Yes instance

$$\begin{cases} x_1x_2 + x_3 = 1 \\ x_1 + x_2x_3 = 1 \\ x_1x_3 + x_2 = 1 \end{cases}$$

$\mathbf{x} = (1, 1, 0)$. ✓

No instance

$$\begin{cases} x_1x_2 = 1 \\ x_1 + x_2 = 1 \end{cases}$$

1st forces $x_1 = x_2 = 1$,
then $1 + 1 = 0 \neq 1$. ×

QUADEQ Is NP-Complete

Theorem 6
QUADEQ *is NP-complete.*

- $\mathrm{QUADEQ} \in \mathrm{NP}$: certificate $\mathbf{x} \in \{0,1\}^n$.
- NP-hard: show $3\mathrm{SAT} \leq_p \mathrm{QUADEQ}$.

Arithmetize over $\mathrm{GF}(2)$: $x_i \mapsto x_i$, $\neg x_i \mapsto 1 + x_i$.

$\ell_1 \vee \ell_2 \vee \ell_3$ is false $\iff (1+\ell_1)(1+\ell_2)(1+\ell_3) = 1$.

$3\mathrm{SAT} \leq_p \mathrm{QUADEQ}$: Construction

For each clause $C_j = \ell_1 \vee \ell_2 \vee \ell_3$, add a fresh y_j and two equations:

$$\boxed{y_j = (1+\ell_1)(1+\ell_2), \qquad y_j\,(1+\ell_3) = 0.}$$

Example: $C = x_1 \vee \neg x_2 \vee x_3$. Here $1+\ell_1 = 1+x_1$, $1+\ell_2 = x_2$, $1+\ell_3 = 1+x_3$:

$$y = (1+x_1)\,x_2, \qquad y\,(1+x_3) = 0.$$

Correctness

Claim: σ satisfies $C_j \iff \sigma$ extends to y_j satisfying both equations.

- The 1st equation forces $y_j = (1 + \ell_1)(1 + \ell_2)$.
- Then the 2nd equation says $(1 + \ell_1)(1 + \ell_2)(1 + \ell_3) = 0$, i.e., some $\ell_i = 1$, i.e., C_j is satisfied.

Putting it together: $n + m$ variables, $2m$ quadratic equations.

$$\varphi \in \mathrm{3SAT} \iff E \in \mathrm{QUADEQ}. \qquad \square$$

SUBSET-SUM

Definition 7
Given integers $a_1, \ldots, a_k$ and a target t, written in binary:

$$\text{SUBSET-SUM} = \{ \langle a_1, \ldots, a_k, t \rangle : \exists S \subseteq [k],\ \textstyle\sum_{i \in S} a_i = t \}.$$

Example: $2, 3, 6, 8, 10$.

- $t = 15$: $2 + 3 + 10 = 15$. ✓
- $t = 22$: no subset works. ×

Theorem 8
SUBSET-SUM *is NP-complete.*

$3\text{SAT} \leq_p \text{SUBSET-SUM}$: Construction

Decimal numbers, one digit per **variable** and per **clause**.

- v_i / v'_i (for x_i / $\neg x_i$): digit x_i is 1; digit C_j is 1 if the literal is in C_j.
- s_j, s'_j: digit C_j is 1.
- t: 1 on variable digits, 3 on clause digits.

$$\varphi = \underbrace{(x_1 \vee \neg x_2 \vee x_3)}_{C_1} \wedge \underbrace{(\neg x_1 \vee x_2 \vee x_3)}_{C_2}$$

	x_1	x_2	x_3	C_1	C_2
v_1	1	0	0	1	0
v'_1	1	0	0	0	1
v_2	0	1	0	0	1
v'_2	0	1	0	1	0
v_3	0	0	1	1	1
v'_3	0	0	1	0	0
s_1, s'_1	0	0	0	1	0
s_2, s'_2	0	0	0	0	1
t	1	1	1	3	3

Correctness

No carries: each column sums to at most 5.

- **Variable digit** $= 1$**:** pick exactly one of v_i, v'_i. This is an assignment.
- **Clause digit** $= 3$**:** true literals give 0–3, slacks add 0–2.
  Reach 3 $\iff$ at least one true literal.

So

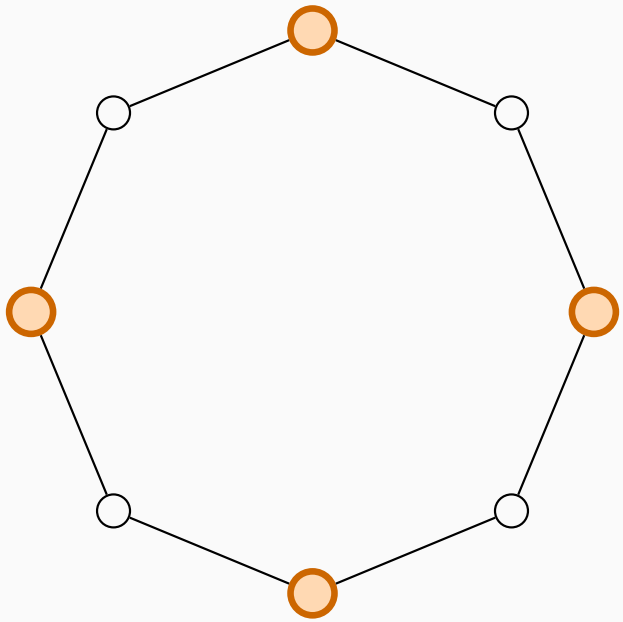
$\varphi \in \mathrm{3SAT} \iff \langle v, v', s, s', t \rangle \in \mathrm{SUBSET\text{-}SUM}$.

$2n + 2m$ numbers, $n + m$ digits. $\square$

Independent Set: Definition

Independent set: a set of vertices, no two joined by an edge.

$$\text{INDSET} = \{ \langle G, k \rangle : G \text{ has an ind. set of size } k \}$$



Orange vertices: an independent set of size 4.

3SAT $\leq_p$ INDSET: Construction

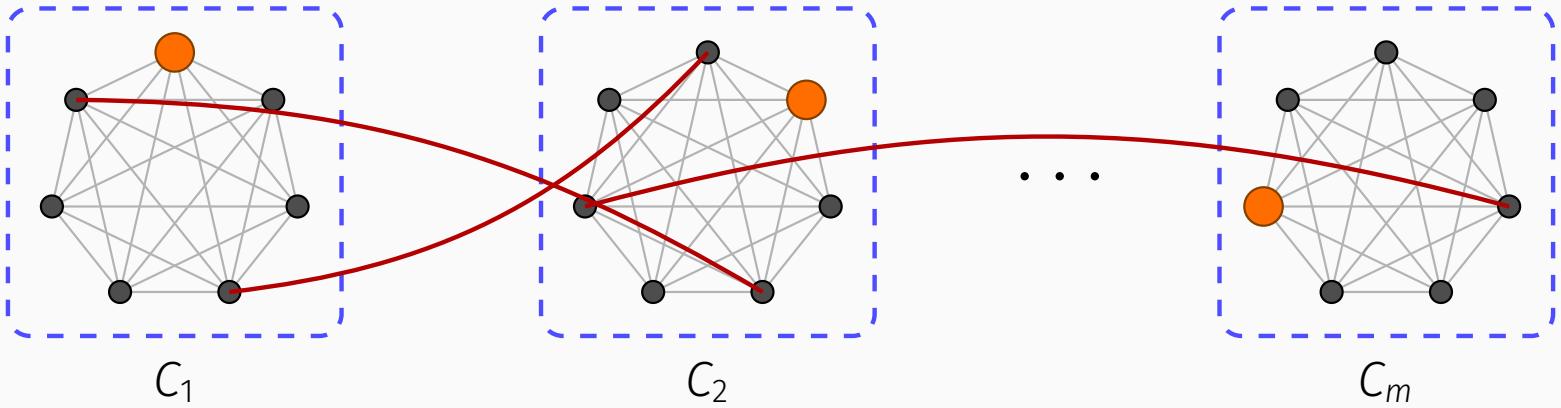
Theorem 9
$3\mathrm{SAT} \leq_p \mathrm{INDSET}$.

Given a 3CNF φ with m clauses, build G:

- Each clause $\rightarrow$ **7 vertices**, one per satisfying assignment of its 3 variables.
- The 7 vertices form a K_7.

Goal: $\varphi \in 3\mathrm{SAT} \iff \langle G, m \rangle \in \mathrm{INDSET}$.

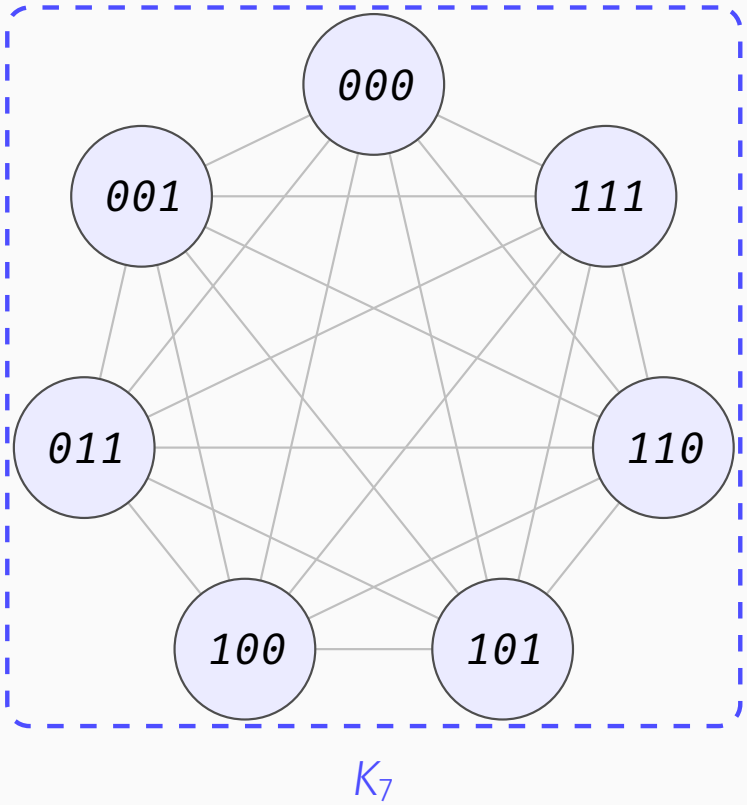
3SAT $\leq_p$ INDSET: The Construction, Pictured



- Dashed: one K_7 per clause, a vertex per satisfying assignment.
- Red: two assignments that disagree on a shared variable.
- Orange: one per cluster, no red edges $=$ satisfying σ.

Example: One Clause's Cluster

Clause $C = x_1 \vee \neg x_2 \vee x_3$, vertices labeled by (x_1, x_2, x_3):

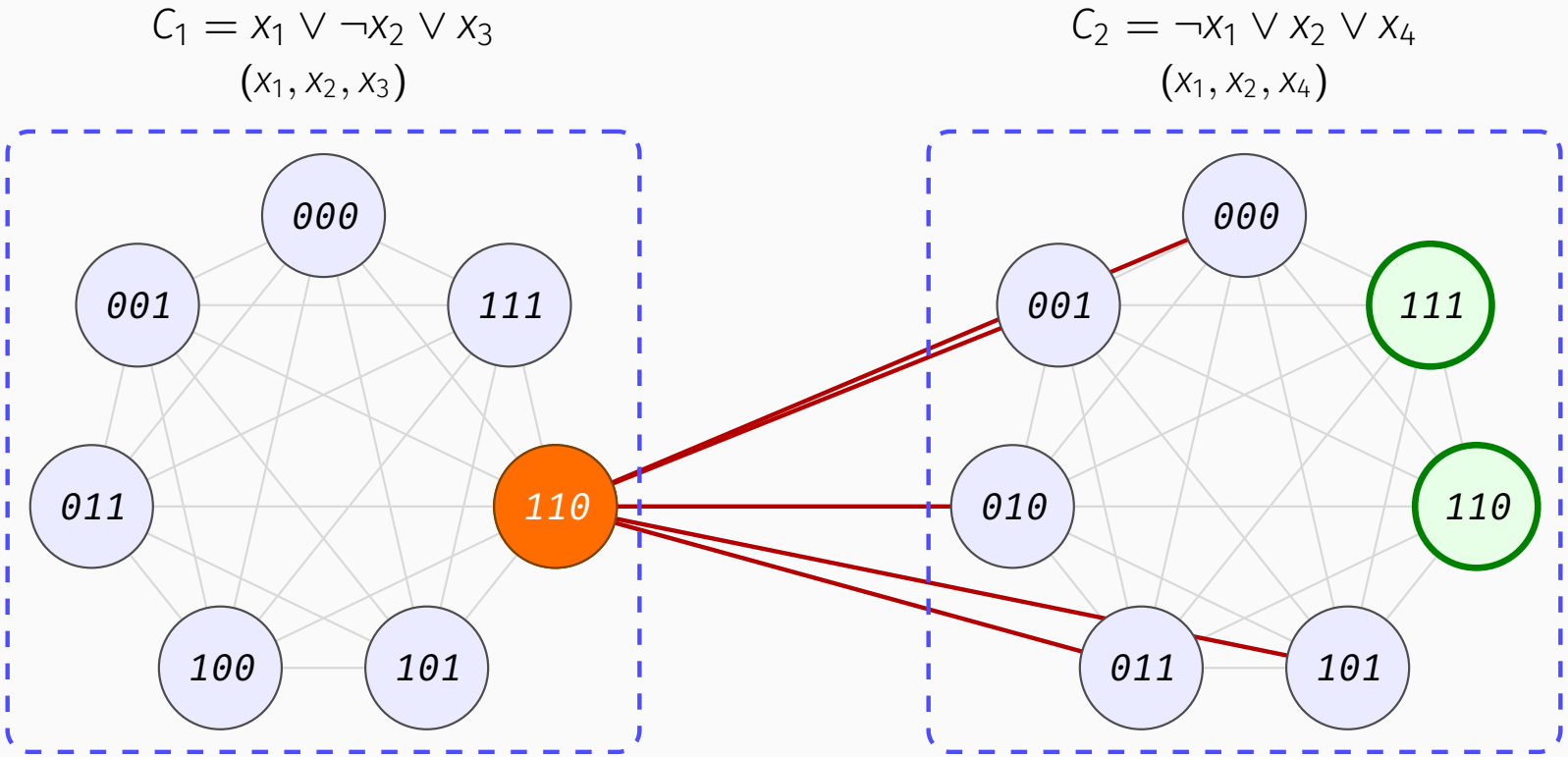




falsifies C
excluded

Cross-Cluster Edges: Enforcing Consistency

Join u, v in different clusters if they **disagree on a shared variable**.



Shared: x_1, x_2. 110 ($x_1 = x_2 = 1$) is joined to every vertex of C_2 with $(x_1, x_2) \neq (1, 1)$; only 110, 111 are compatible.

Proof: $\varphi \in \mathrm{3SAT} \implies \langle G, m \rangle \in \mathrm{INDSET}$

Let σ satisfy φ.

- In each cluster, pick the vertex $\sigma|_{C_j}$ (restriction of σ to C_j's variables).
- One per cluster $\implies$ no K_7 edges.
- All from the same σ $\implies$ consistent, no cross edges.

So these m vertices form an independent set.

Proof: $\langle G, m \rangle \in \mathrm{INDSET} \implies \varphi \in \mathrm{3SAT}$

Let S be an independent set of size m.

- Each cluster is a $K_7 \implies$ at most one vertex of S per cluster.
- $|S| = m \implies$ exactly one per cluster.
- No cross edges $\implies$ chosen local assignments are consistent.

Glue them into σ; each clause is satisfied by its own vertex.

Clique

$$\text{CLIQUE} = \{ \langle G, k \rangle : G \text{ has a clique of size } k \}$$

Theorem 10
$\text{INDSET} \leq_p \text{CLIQUE}$.

Proof: Map $\langle G, k \rangle \mapsto \langle \overline{G}, k \rangle$. Then

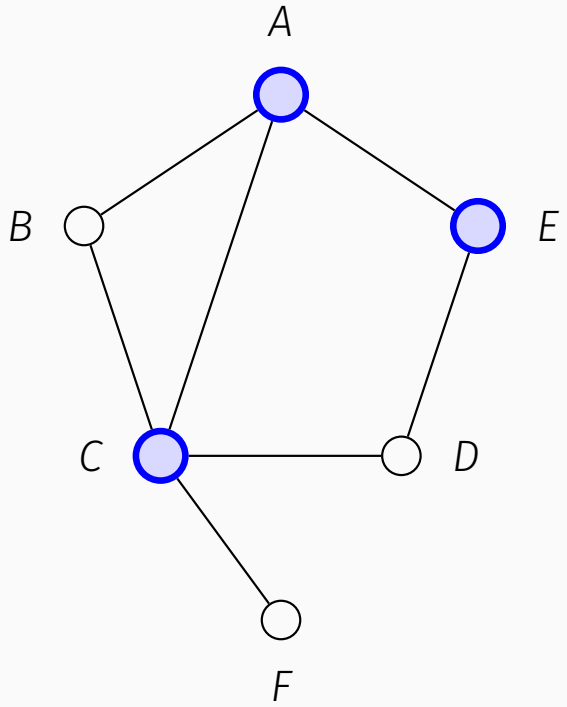
S is independent in $G \iff S$ is a clique in $\overline{G}$.

□

Vertex Cover

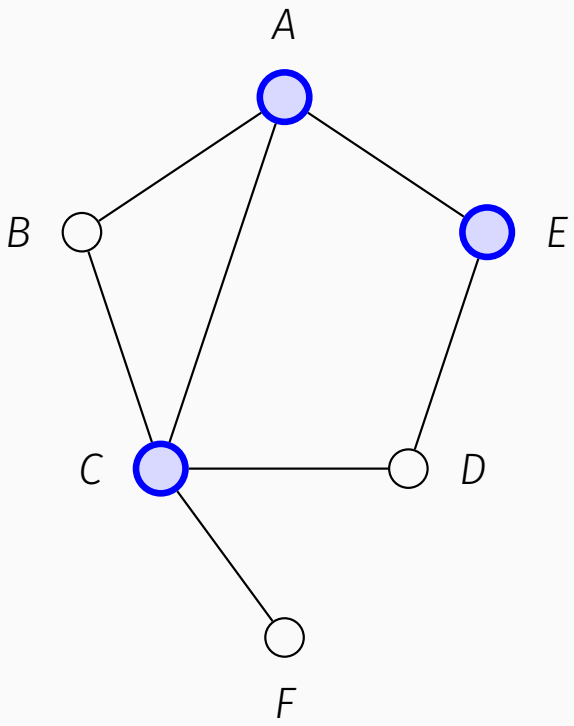
A **vertex cover**: a set of vertices touching every edge.

Example: $\{A, C, E\}$.

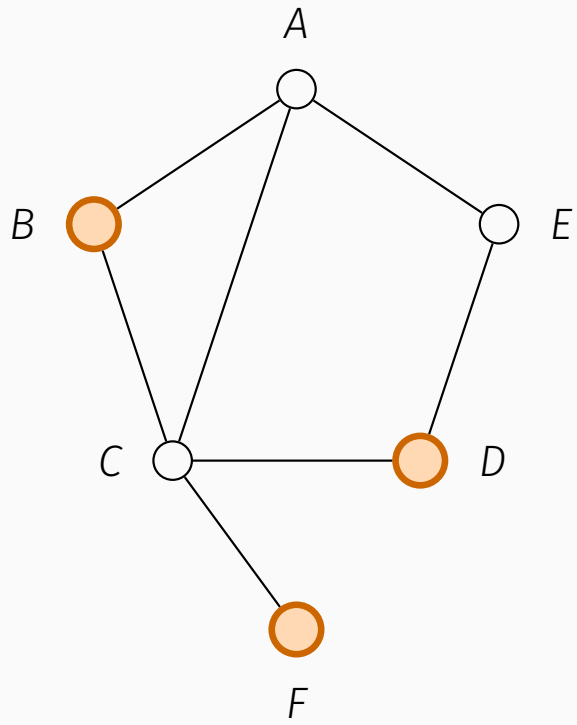


$$\text{VERTEX-COVER} = \{ \langle G, k \rangle : G \text{ has a size-}k \text{ vertex cover} \}$$

Vertex Cover and Independent Set: Complements



Vertex cover $\{A, C, E\}$



Independent set $\{B, D, F\}$

S is a vertex cover $\iff$ $V \setminus S$ is independent

INDSET $\leq_p$ VERTEX-COVER

Theorem 11
$\mathrm{INDSET} \leq_p \mathrm{VERTEX\text{-}COVER}$.

Proof: Let $n = |V(G)|$. Map $\langle G, k \rangle \mapsto \langle G, n - k \rangle$.

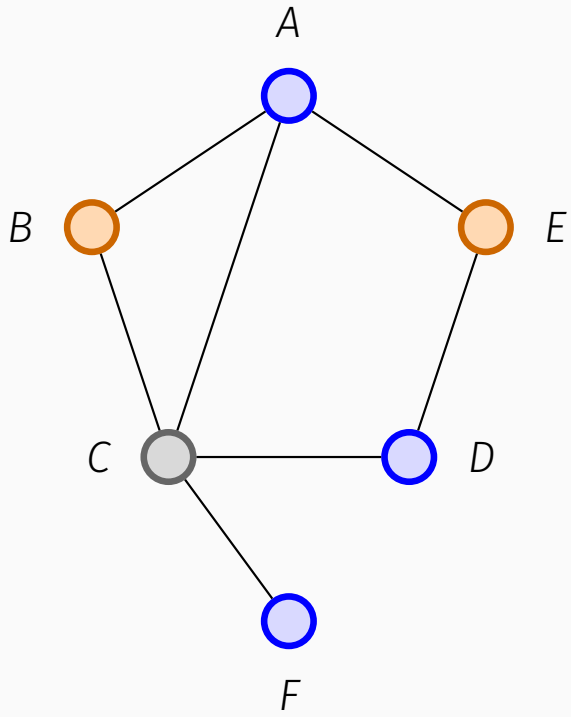
Then

$$S \text{ is independent} \iff V \setminus S \text{ is a vertex cover.}$$

□

3-Coloring

A **3-coloring**: color each vertex with one of 3 colors, so that adjacent vertices get different colors.



$$3\mathrm{COL} = \{\, \langle G \rangle : G \text{ has a 3-coloring} \,\}$$

3COL Is NP-Complete

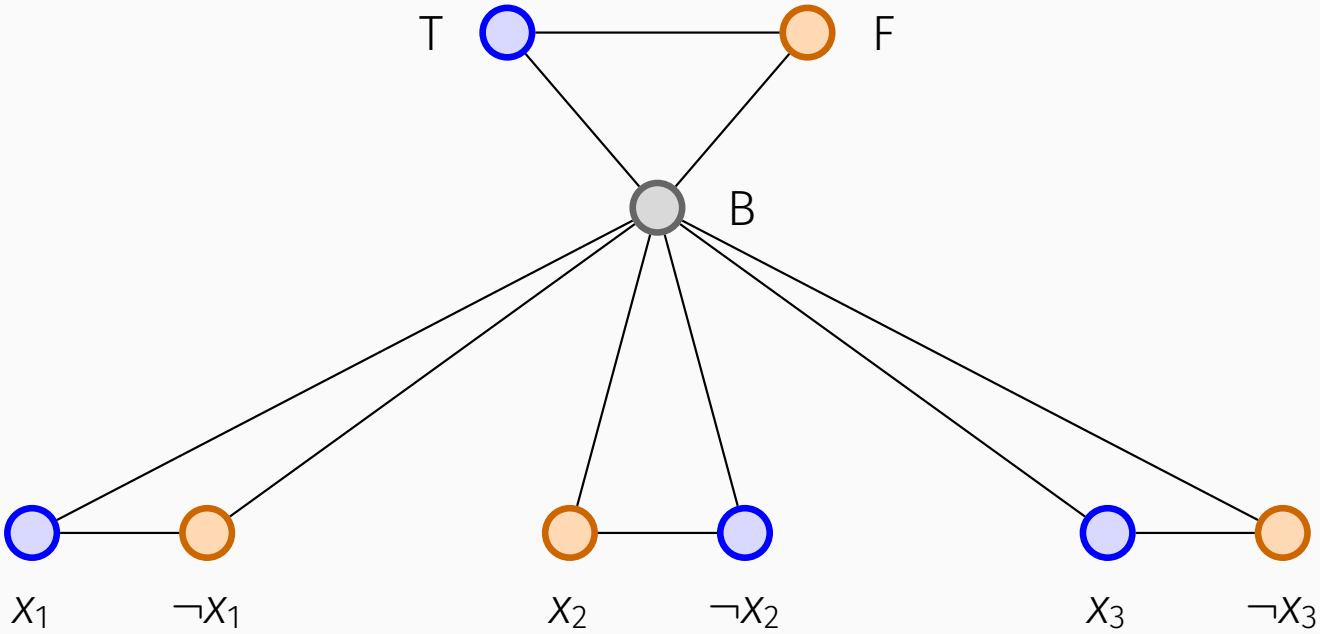
Theorem 12
$3\mathrm{COL}$ *is NP-complete.*

- $3\mathrm{COL} \in \mathrm{NP}$: the certificate is a coloring.
- $3\mathrm{SAT} \leq_p 3\mathrm{COL}$: from a 3-CNF φ, build G with

$$\varphi \text{ satisfiable} \iff G \text{ is 3-colorable.}$$

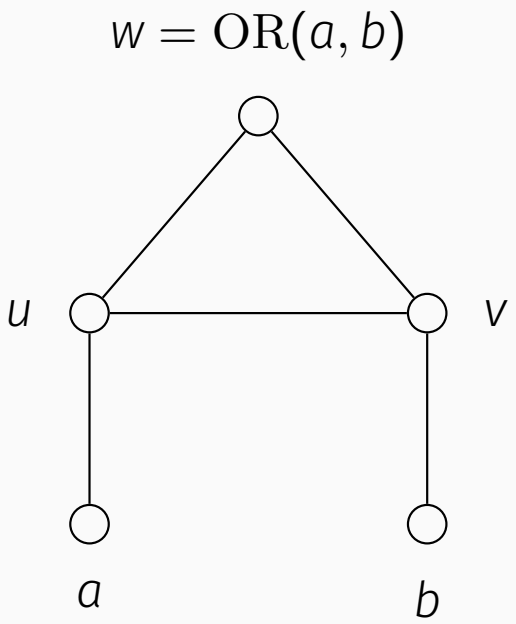
Three gadgets: **palette**, **variables**, **clauses**.

Palette and Variable Gadgets



- Palette: a triangle, so its colors are distinct; call them T, F, B.
- Each triangle x_i, $\neg x_i$, B: one of x_i, $\neg x_i$ is T, the other F.

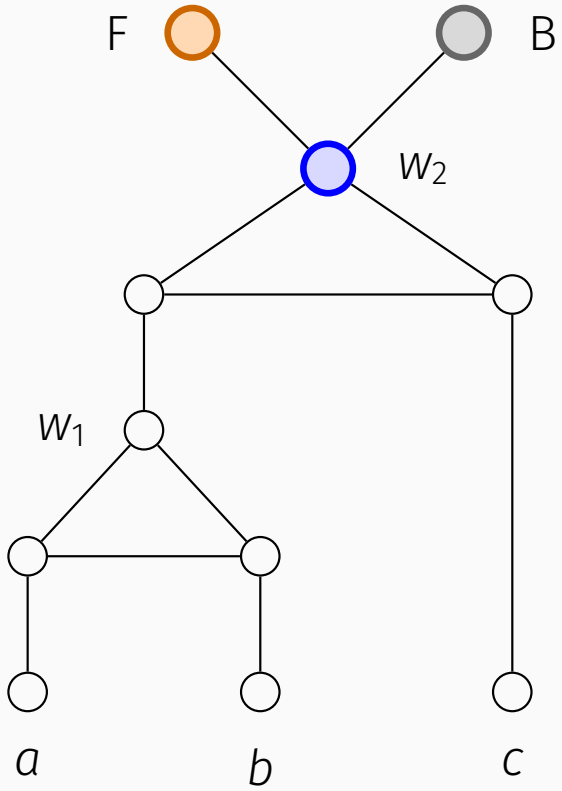
The OR Gadget



a	b	can $w =$ T?
F	F	no (w forced to F)
F	T	yes
T	F	yes
T	T	yes

$$w \text{ can be T} \iff a = \text{T or } b = \text{T}$$

The Clause Gadget



For each clause $(a \vee b \vee c)$:

- $w_1 = \mathrm{OR}(a, b)$
- $w_2 = \mathrm{OR}(w_1, c)$
- w_2 adjacent to F and B, so $w_2 = \mathsf{T}$

Colorable $\iff$ some literal is T.

Correctness

($\Rightarrow$) Satisfying assignment: color x_i by its value. Each clause has a true literal, so its gadget can be colored with $w_2 = \mathsf{T}$.

($\Leftarrow$) 3-coloring: rename colors so the palette is T, F, B. Read off x_i from its color. Each $w_2 = \mathsf{T}$, so each clause has a true literal.

Size: $3 + 2n + 6m$ vertices, built in poly time. $\square$

Not-All-Equal SAT

An assignment **NAE-satisfies** a CNF if every clause has at least one TRUE literal and at least one FALSE literal.

$\mathrm{NAE\text{-}3SAT} = \{ \langle \varphi \rangle : \text{3-CNF } \varphi \text{ is NAE-satisfiable} \}$

Symmetry: if σ NAE-satisfies φ, so does its complement $\overline{\sigma}$.

3SAT $\leq_p$ NAE-4SAT

Add one new variable s to every clause:

$$(a \vee b \vee c) \longmapsto \mathrm{NAE}(a, b, c, s).$$

($\Rightarrow$) Set $s = \mathrm{FALSE}$. Each clause has a true literal, so not all equal.

($\Leftarrow$) By symmetry, assume $s = \mathrm{FALSE}$. Not all of a, b, c, s are FALSE, so some of a, b, c is TRUE.

NAE-4SAT $\leq_p$ NAE-3SAT

Split each clause with a new variable w:

$$\mathrm{NAE}(a, b, c, s) \longmapsto \mathrm{NAE}(a, b, w) \wedge \mathrm{NAE}(\neg w, c, s)$$

($\Rightarrow$) If $a \neq b$, set $w = c$; if $a = b$, set $w = \neg a$.

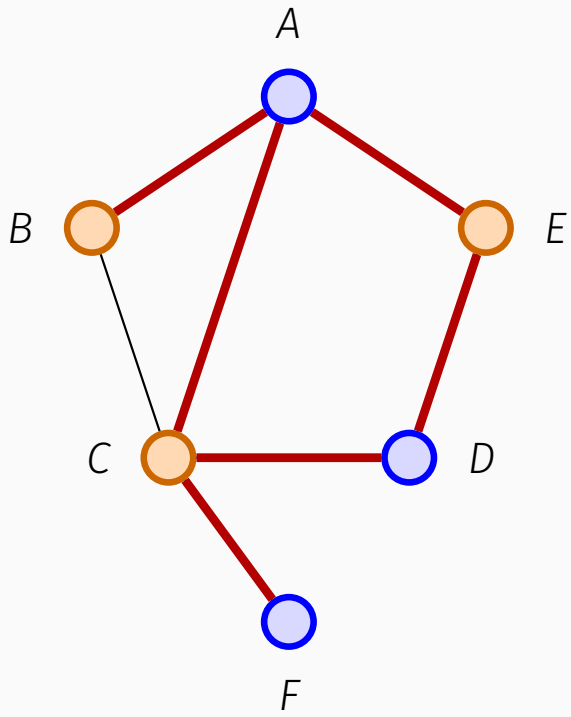
($\Leftarrow$) If $a = b = c = s$, then $w = \neg a$ and $\neg w = a$, so $\mathrm{NAE}(\neg w, c, s)$ fails.

Theorem 13
$\mathrm{NAE\text{-}3SAT}$ *is NP-complete.*

Max-Cut

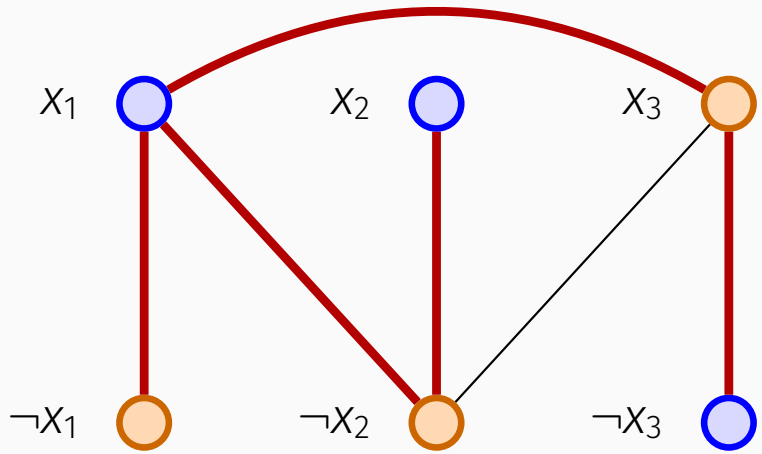
A **cut** $(S, V \setminus S)$: its size is the number of edges between S and $V \setminus S$.

Example: $S = \{A, D, F\}$ cuts 6 of 7 edges.



$$\text{MAX-CUT} = \{ \langle G, k \rangle : G \text{ has a cut of size} \geq k \}$$

NAE-3SAT $\leq_p$ MAX-CUT: Construction



$\varphi = (x_1 \vee \neg x_2 \vee x_3)$, $k = 5$

blue: true literals; red: cut edges

For n variables, m clauses:

- Vertices: all $2n$ literals.
- m_i parallel edges between x_i and $\neg x_i$, where m_i = #occurrences of x_i.
- Each clause: a triangle on its 3 literals.
- $k = 5m$.

NAE-3SAT $\leq_p$ MAX-CUT: Correctness

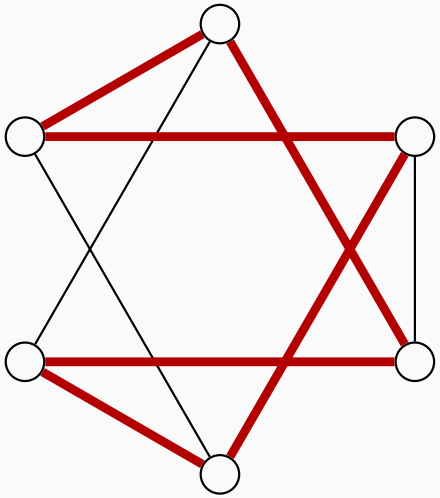
Variable edges: $\sum_i m_i = 3m$. A triangle has at most 2 cut edges, and exactly 2 iff its vertices are not all on one side.

($\Rightarrow$) S = true literals: all $3m$ variable edges and 2 edges per triangle are cut, total $5m$.

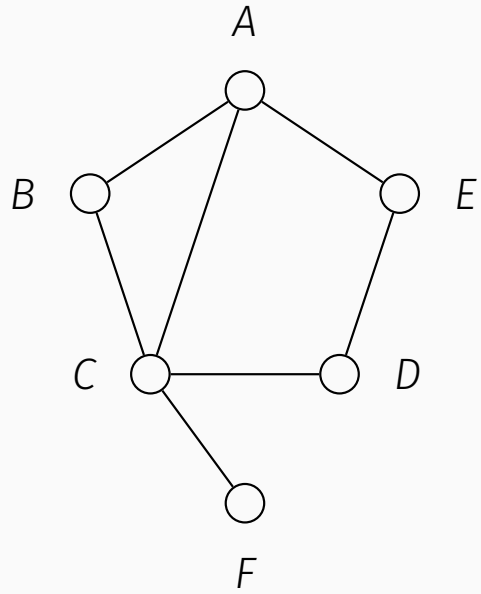
($\Leftarrow$) A cut of size $\geq 5m$ must cut **all** variable edges ($x_i, \neg x_i$ on opposite sides: a valid assignment) and 2 edges of **every** triangle (each clause NAE-satisfied). $\square$

Hamiltonian Cycle

A **Hamiltonian cycle**: a cycle visiting every vertex exactly once.



Yes ✓



No: F has degree 1. ×

$$\mathrm{HAMCYCLE} = \{\, \langle G \rangle : G \text{ has a Hamiltonian cycle} \,\}$$

HAMCYCLE Is NP-Complete

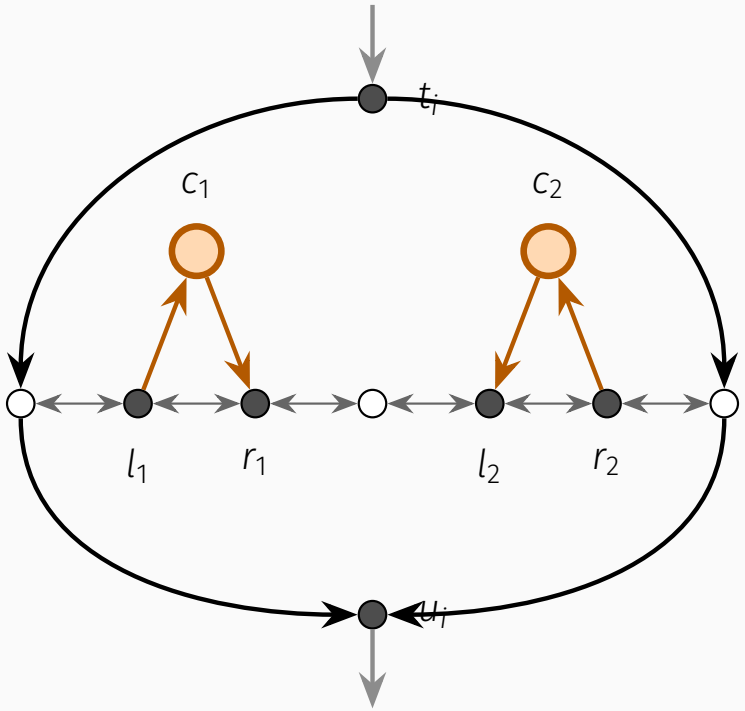
Theorem 14
$\mathrm{HAMCYCLE}$ *is NP-complete.*

- $\mathrm{HAMCYCLE} \in \mathrm{NP}$: certificate is the cycle.
- NP-hard, in two steps:

$$3\mathrm{SAT} \leq_p \mathrm{DHAMCYCLE} \leq_p \mathrm{HAMCYCLE}.$$

DHAMCYCLE: the same question on **directed** graphs.

$3\text{SAT} \leq_p \text{DHAMCYCLE}$: Gadgets



- Variable x_i: a diamond; row of $3m + 1$ vertices, a pair (l_j, r_j) per clause.
- Left→right = TRUE, right→left = FALSE.
- Clause C_j: vertex c_j. $x_i \in C_j$: $l_j \to c_j \to r_j$; $\neg x_i \in C_j$: $r_j \to c_j \to l_j$.

$3\mathrm{SAT} \leq_p \mathrm{DHAMCYCLE}$: The Whole Graph

- Chain the diamonds: $u_i \to t_{i+1}$, and $u_n \to t_1$.
- Add $c_1, \ldots, c_m$ with their detour edges.
- Size: $n(3m + 3) + m$ vertices.

Intuition: the cycle sweeps $x_1, \ldots, x_n$ in turn; each c_j must be picked up by a detour from a row going the "right" way.

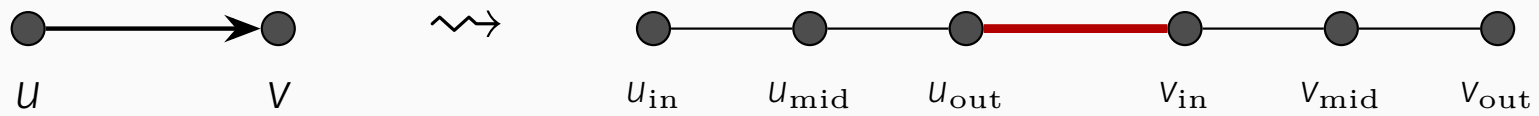
$3\mathrm{SAT} \leq_p \mathrm{DHAMCYCLE}$: Correctness

($\Rightarrow$) Given satisfying σ: sweep row i left$\rightarrow$right iff $\sigma(x_i) = \mathrm{TRUE}$. Each C_j has a true literal; take the detour through c_j there.

($\Leftarrow$) Given a Hamiltonian cycle:

- Each detour enters c_j from one vertex of a pair and returns to the **other vertex of the same pair**; otherwise a separator gets stranded.
- Skip the detours: each row is swept in one direction $\Rightarrow \sigma$.
- c_j is picked up by a row going the right way $\Rightarrow C_j$ satisfied. $\square$

DHAMCYCLE $\leq_p$ HAMCYCLE



- Each $v \mapsto$ path $v_{\text{in}} - v_{\text{mid}} - v_{\text{out}}$; each arc $u \to v \mapsto$ edge $u_{\text{out}} - v_{\text{in}}$.
- v_{mid} has degree 2 $\Rightarrow$ any Hamiltonian cycle goes in–mid–out.
- So it reads off a directed Hamiltonian cycle, and vice versa. □

Traveling Salesman

Complete graph K_n, weights $w(e) \in \mathbb{N}$, budget B:

$$\mathrm{TSP} = \{ \langle K_n, w, B \rangle : \exists \text{ a tour of total weight} \leq B \}$$

Theorem 15
TSP *is NP-complete.*

Proof: $\in$ NP: certificate is the tour. $\mathrm{HAMCYCLE} \leq_p \mathrm{TSP}$:

$$w(e) = \begin{cases} 1 & e \in E(G) \\ 2 & e \notin E(G) \end{cases}, \qquad B = n.$$

Tour of weight $n \iff$ uses only edges of $G \iff$ Hamiltonian cycle in G. $\square$

More Classic NP-Complete Problems

Problem	Question
CIRCUIT-SAT	Is a Boolean circuit satisfiable?
1-IN-3 SAT	Exactly one true literal in each clause?
MAX-2SAT	Can $\geq k$ clauses of a 2-CNF be satisfied?
HAMPATH	A path visiting every vertex exactly once?
DOMINATING-SET	k vertices dominating all others?
FEEDBACK-VERTEX-SET	k vertices hitting every cycle?
STEINER-TREE	A tree with $\leq k$ edges spanning given terminals?
3D-MATCHING	Disjoint triples covering $X \cup Y \cup Z$?
SET-COVER	k subsets covering the universe?
PARTITION	Split numbers into two halves of equal sum?

... and thousands more

NP-Intermediate Problems

Not everything in NP is known to be NP-complete or in P:

- **Graph Isomorphism**: conjectured to be neither in P nor NP-complete.
- **Factoring**: also conjectured not to be in P — and also not believed NP-complete.
- **Discrete Logarithm**: same story as Factoring — believed hard, not believed NP-complete.

Ladner's Theorem

Theorem 16 (Ladner, 1975)
If $\mathrm{P} \neq \mathrm{NP}$*, then some* $L \in \mathrm{NP}$ *is neither in* P *nor NP-complete.*

Such languages are called **NP-intermediate**.

Proof idea: pad SAT so that it is "too sparse" to be NP-complete, yet "too dense" to be in P.

Summary

- **NP-complete**: 3SAT, ILP, INDSET, CLIQUE, VERTEX-COVER, 3COL, NAE-3SAT, MAX-CUT, ..., and thousands more.
- **NP-intermediate**: in NP, but neither in P nor NP-complete. Such problems exist if $\mathrm{P} \neq \mathrm{NP}$ (Ladner).
- **Candidates**: Factoring and Discrete Log, the foundation of today's (pre-quantum) public-key cryptography.

P, NP and NP-Complete

